

Educational Chatbot for QAs

Hybrid (Elasticsearch + RAG) Approach

Name	Email	Role (leader, UI, eval, etc.)
Chaitanya Datta Maddukuri	cmaddukuri@hawk.illinois tech.edu	Leader, UI, Data collection and chunking
Harish Namasivayam Muthuswamy	hmuthuswamy@hawk.illino istech.edu	Data collection, Preprocessing and chunking
Karthik Kolli	kkolli1@hawk.illinoiste ch.edu	System Architecture, Backend Development, Evaluation and Documentation
Ananya Bhoolam Praveen	bananya@hawk.illinoist ech.edu	System Architecture, Backend Development, Evaluation and Documentation

Abstract

University information systems often distribute critical academic data including policies, tuition rates, calendars, and faculty directories, across fragmented sources and formats, making it difficult for students to retrieve accurate answers efficiently. Existing LLM-based chatbot approaches offer conversational access but risk generating hallucinated or factually unsupported responses, particularly for queries involving specific deadlines, dollar amounts, or policy rules. To address this, we present a Hybrid University Information Chatbot for Illinois Institute of Technology (IIT) that combines Elasticsearch for precise, source-grounded document retrieval with GPT-4o for natural language answer generation. The system employs a GPT-4o-based intent router that classifies queries across four specialized domains academic policies, tuition and fees, academic calendar, and department contacts, each backed by a uniform retrieval pipeline combining BM25 lexical search and kNN vector search fused via Reciprocal Rank Fusion (RRF), followed by cross-encoder reranking. Domain-specific slot-filling and query expansion are applied before retrieval to resolve ambiguity and improve recall. A multi-turn clarification dialog handles queries that remain underspecified after slot extraction.

The system is built on 44 official IIT web sources indexed into domain-specific Elasticsearch indices. Evaluation across 89 questions spanning 9 categories, including registration, financial aid, transcripts, and multi-topic queries using Recall, Precision, and Hit Rate shows hit rates between 81% and 100% across all categories. These results demonstrate that domain-specialized hybrid retrieval paired with grounded LLM generation produces accurate, explainable, and hallucination-resistant responses. The architecture is modular and extensible, making it applicable to other academic institutions with minimal adaptation.

1. Introduction & Motivation

1.1 Background and Motivation

Students at Illinois Institute of Technology regularly need quick access to accurate information about tuition fees, semester deadlines, course registration procedures, academic policies, and faculty contact details. This project focuses on four core information domains: academic policies, tuition and fees, the academic calendar, and department contacts. This information is officially available but spread across dozens of web pages, PDF handbooks, and departmental portals. A student trying to find the last day to drop a course, or the tuition rate for a specific graduate program, may need to navigate several unrelated pages before finding a definitive answer. This creates unnecessary friction in routine academic decision-making and increases the risk of students acting on outdated or misremembered information.

Purely LLM-based chatbots have emerged as one solution to this problem, offering students a conversational interface for academic queries. However, these systems generate responses from parametric knowledge rather than live official documents, which makes them unreliable for time-sensitive or institution-specific information. A chatbot that confidently states an incorrect fee amount, or a wrong registration deadline causes more harm than no chatbot at all. The goal of this project is to build a system that retrieves answers directly from authoritative IIT sources and uses a language model only to express those retrieved facts in clear, natural language, ensuring both accuracy and accessibility.

1.2 Proposed Approach

This project develops a Hybrid University Information Chatbot that combines Elasticsearch with Retrieval-Augmented Generation (RAG). Official IIT documents are indexed into domain-specific Elasticsearch indices covering the four domains described above. When a student submits a query, the system routes it to the appropriate domain using a GPT-4o intent classifier, applies slot-filling and query expansion to sharpen the search, and retrieves candidate documents using a pipeline that combines BM25 lexical search with kNN vector search, fused via Reciprocal Rank Fusion and reranked by a cross-encoder model. The language model is invoked only to generate the final response from the retrieved content, not to recall facts independently. This design keeps the LLM responsible for fluency and the retrieval system responsible for factual accuracy, which reduces hallucination and lowers the per-query computational cost compared to systems where the LLM performs both retrieval and generation.

2. Data Sources

Category	Source	Format	Size	Notes
Calendar	iit.edu/registrar/academic-calendar	HTML	-	Semester dates, registration deadlines, and key milestones for Spring, Summer, and Fall 2026
Calendar	iit.edu/registrar/academic-calendar/subsequent-academic-years	HTML	-	Academic calendar events for the 2026-27 year
Calendar	iit.edu/coursera/coursera-academic-calendar	HTML	-	Term dates for Coursera Spring 2026 (Terms A and B) and Summer 2026
Calendar	iit.edu/hr/employee-resources/paid-time	HTML	-	University paid holidays for the 2026-27 calendar year
Contacts	iit.edu/directory	HTML	-	Full IIT staff and faculty directory; 1,552 records covering all department
Tuition and Fees	https://kentlaw.iit.edu/law/admissions/financing-your-education/tuition-fees/llm-tuition-and-fees	HTML	-	Chicago-Kent College of Law: LLM programme tuition and fee schedule
Tuition and Fees	https://www.iit.edu/student-accounting/tuition-and-fees/current-tuition-and-fees/current-institute-design	HTML	-	Institute of Design graduate tuition and fee schedule

Tuition and Fees	https://www.iit.edu/student-accounting/tuition-and-fees/current-tuition-and-fees/current-intensive-english-program	HTML	-	Intensive English Program tuition and fee schedule
Tuition and Fees	https://www.iit.edu/student-accounting/tuition-and-fees/current-tuition-and-fees/current-mies-campus-graduate	HTML	-	Mies Campus graduate tuition and fee schedule
Tuition and Fees	https://www.iit.edu/student-accounting/tuition-and-fees/current-tuition-and-fees/current-mies-campus-undergraduate	HTML	-	Mies Campus undergraduate tuition and fee schedule
Tuition and Fees	https://www.iit.edu/student-accounting/tuition-and-fees/current-tuition-and-fees/current-stuart-school-business	HTML	-	Stuart School of Business tuition and fee schedule
Documents	https://www.iit.edu/sites/default/files/2026-02/2025-2026%20Student%20Handbook%20Final%20Copy_0.pdf	PDF	188 Pages	Undergraduate academic policies and procedures; primary long-form reference
Documents	https://webmaster.iit.edu/files/graduate-academic-affairs/co-terminal-student-handbook.pdf	PDF	13 Pages	Policies governing co-terminal (combined bachelor's/master's) degree students
Documents	https://catalog.iit.edu/graduate/academic-policies-procedures/grades-transcripts/	HTML	-	Graduate academic policies on grading and transcripts
Documents	https://www.iit.edu/commencement	HTML	-	Commencement ceremony overview
Documents	https://www.iit.edu/commencement/event-details-and-schedules	HTML	-	Commencement event schedule and logistical details

Documents	https://www.iit.edu/coursera	HTML	-	Coursera at IIT programme overview
Documents	https://www.iit.edu/directory?organization_type=16	HTML	-	Registrar office staff directory listing
Documents	https://www.iit.edu/financial-aid/policies-and-procedures/accelerated-masters-program-student-policy	HTML	-	Financial aid policy for the accelerated master's programme
Documents	https://www.iit.edu/registrar/important-information	HTML	-	Notices and important registration information
Documents	https://www.iit.edu/registrar/people	HTML	-	Registrar office staff contacts
Documents	https://www.iit.edu/registrar/policies-and-procedures	HTML	-	Top-level registrar policies index
Documents	https://www.iit.edu/registrar/policies-and-procedures/compressed-term-courses	HTML	-	Policies for compressed-term course enrollment
Documents	https://www.iit.edu/registrar/policies-and-procedures/contact-and-credit-hours	HTML	-	Definitions of contact hours and credit hours
Documents	https://www.iit.edu/registrar/policies-and-procedures/course-cancellation-policy	HTML	-	Policy governing course cancellation by the university
Documents	https://www.iit.edu/registrar/policies-and-procedures/physical-or-financial-hardship-withdrawal-policy	HTML	-	Hardship withdrawal policy and eligibility

Documents	https://www.iit.edu/registrar/policies-and-procedures/withdrawing-vs-dropping-course	HTML	-	Distinction between a course withdrawal and a drop
Documents	https://www.iit.edu/registrar/registration	HTML	-	Registration overview and links
Documents	https://www.iit.edu/registrar/registration/advising	HTML	-	Academic advising and registration guidance
Documents	https://www.iit.edu/registrar/registration/auditing-course	HTML	-	Policy for auditing a course without credit
Documents	https://www.iit.edu/registrar/registration/course-repeat-policy	HTML	-	Course repeat and grade replacement rules
Documents	https://www.iit.edu/registrar/registration/credit-hour-limits	HTML	-	Per-term credit hour enrollment limits
Documents	https://www.iit.edu/registrar/registration/full-time-status	HTML	-	Full-time enrollment status definitions.
Documents	https://www.iit.edu/registrar/registration/hold-information	HTML	-	Registration holds, types, and resolution
Documents	https://www.iit.edu/registrar/registration/internet-course-registration	HTML	-	Online course registration procedures
Documents	https://www.iit.edu/registrar/registration/late-registration-request	HTML	-	Process for requesting late registration

Documents	https://www.iit.edu/registrar/registration/registration-changes-add-drop-w-grades	HTML	-	Add, drop, and W-grade change procedures
Documents	https://www.iit.edu/registrar/registration/undergraduate-approval-graduate-course	HTML	-	Process for undergraduates enrolling in graduate courses
Documents	https://www.iit.edu/registrar/registration/withdrawal-course	HTML	-	Course withdrawal procedures and deadlines
Documents	https://www.iit.edu/registrar/students-and-alumni	HTML	-	Student and alumni registrar services overview
Documents	https://www.iit.edu/registrar/students-and-alumni/grade-legend	HTML	-	Grade scale and symbol definitions
Documents	https://www.iit.edu/registrar/students-and-alumni/preferred-first-name-policy	HTML	-	Policy for updating preferred first name on records
Documents	https://www.iit.edu/registrar/students-and-alumni/transcripts	HTML	-	Official transcript request procedures
Documents	https://www.iit.edu/student-accounting/tuition-and-fees/mandatory-and-other-fees	HTML	-	Mandatory and ancillary fee schedules
Documents	https://www.iit.edu/ugaa/services/taking-course-passfail	HTML	-	Pass/fail grading option eligibility and procedures

Table 1: Data Sources Links

2.2 Data Challenges

Preparing data from official university sources for indexing into Elasticsearch was not straightforward. The diversity of source formats and document types across the four domains introduced a distinct set of challenges at every stage of the ingestion pipeline.

2.2.1 Tabular Data Conversion

Tuition fee and calendar information is published as HTML tables on the IIT website. Converting this into a structured JSON format suitable for indexing required identifying column headers correctly and mapping each row to a meaningful record. Misidentified columns or merged cells resulted in data loss or incorrectly labeled fee entries, making this step particularly sensitive to formatting inconsistencies across different school pages.

2.2.2 Noisy Unstructured Documents

Policy documents and student handbooks are long-form PDFs and html pages that contain headers, footers, page numbers, section titles, images, and tables alongside the actual policy text. Extracting only the relevant textual content while discarding boilerplate and non-textual elements required careful preprocessing to avoid injecting noise into the index that would degrade retrieval quality.

2.2.3 Heterogeneous Data Shapes Across Domains

The four domains produced fundamentally different data structures. Calendar data consisted of discrete dated events, tuition data was tabular with numeric fields, policy documents were long-form prose, and contacts were structured directory entries. Each domain required a separate ingestion pipeline with its own parsing, cleaning, and normalization logic rather than a single unified approach.

2.2.4 Missing Information

Directory page embedded role titles, phone numbers, and department labels in free text descriptions rather than structured HTML fields. Extraction rules were written to identify and pull these values from surrounding text. Where extraction produced no result, the field was left blank rather than filled with a fallback value, to avoid indexing incorrect information that could mislead the chatbot's responses.

2.2.5 Outdated Content

Retrieval systems cannot distinguish document recency from relevance. Identifying superseded content required manual source review before indexing. Sources confirmed to reflect current policy were retained; pages that appeared to reference older academic years or deprecated programmes were excluded from the document corpus.

3. System Design

3.1 Architecture Overview

The Chatbot is built as a sequential processing pipeline in which a natural language query from the user passes through a series of individual stages before a response is returned to the user. The pipeline is organized into three layers: A Data Layer consisting of domain specific Elasticsearch indices which is the knowledge base storage method, Retrieval Layer that filters, routes, and retrieves relevant chunks for the query, and a Generation Layer that produces a grounded answer from the retrieved context. The sections that follow describe each layer in turn.

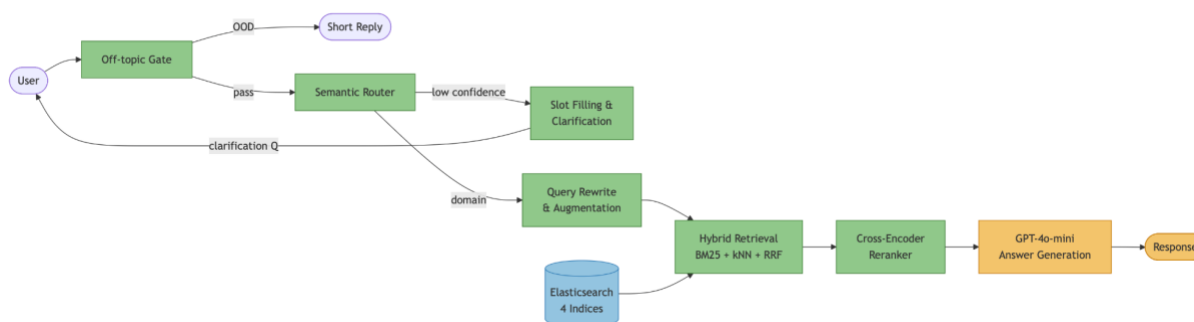


Figure 1: System Pipeline

3.1.1 Data Layer

The system consists of four Elasticsearch indices, each dedicated to a specific domain. The academic calendar index holds documents covering semester dates, registration deadlines, and university events. The contacts index contains records drawn from IIT's staff and faculty directories, each carrying structured fields for name, department, email, phone, and building. The tuition index holds documents covering fee schedules across five schools and three enrollment levels, with structured fields for school, level, fee category, amount, and academic year. The documents index contains document information from university policy pages, covering areas such as academic standing, financial aid, registration procedures, and student conduct.

Each index was assigned its own schema. This separation is a deliberate architectural decision. Due to this separation between fields, it becomes easy to filter fields relevant to a tuition query, school, enrollment level, and fee category, which are meaningless in the context of a contacts query, which requires department and category filters. A shared index would force the schema to accommodate all domains simultaneously, introducing irrelevant fields into every query and reducing retrieval precision. Domain specific indices allow each retrieval path to apply only the filters that are meaningful for that domain through the metadata of each specific domain.

All four indices store dense vector representations of their text content alongside structured metadata. These vectors are generated at index time using the **E5-large-v2 embedding model**, which produces **1024-dimensional representations**. **Cosine similarity** is used for nearest neighbor retrieval across all four indices.

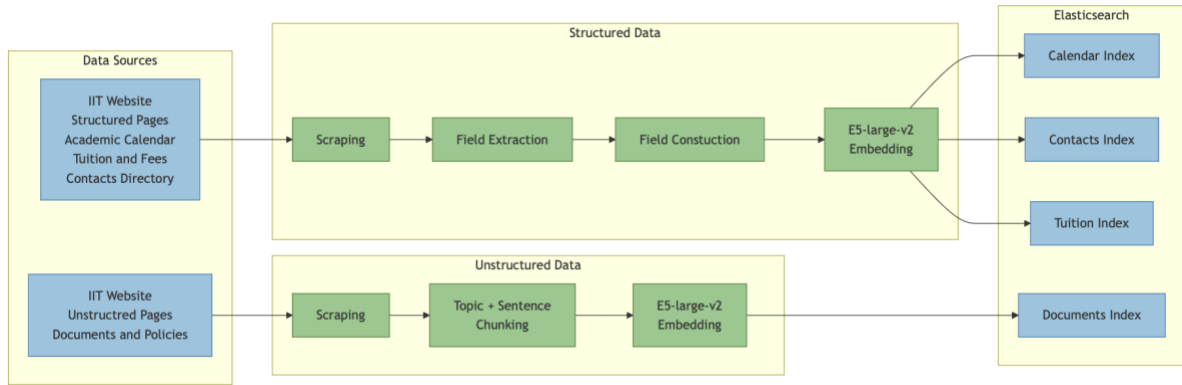


Figure 2: Data Layer

3.1.2 Retrieval Subsystem

The retrieval subsystem spans across stages, starting with query validation and ending with ranked chunks for the generation layer.

Before any embedding or the LLM call is made, an off-topic gate validates the incoming user query against a set of exact match rules and regular expression patterns that identify several things such as greetings, out of scope requests. Queries that clear this filter are passed to a semantic router, which embeds the query using **E5-large-v2** and computes cosine similarity against sets of pre-embedded prototype sentences for each of the four domains. The domain with the highest similarity score is selected if it is above the confidence threshold; queries that fall below the threshold are rejected as out of domain without proceeding further.

Queries assigned to a domain then pass through the slot filling stage that checks whether the query contains the parameters required for that specific domain's retrieval path. A tuition query without a school or enrollment level, for instance, is underspecified across fifteen possible combinations and cannot be resolved without further information. When a required parameter is absent, the system returns a clarification question to the user with an option set that is domain specific.

Once the query is fully specified, the query rewriting step reformulates the user input into a retrieval language that is friendly to the language model using **LLaMA 3.1-8B**. The rewritten query is then expanded with domain specific synonyms before being submitted to the lexical search path. This augmentation is applied only to the BM25 path; the dense retrieval path uses the rewritten query without the synonym expansion as it adds unnecessary noise to the query.

Retrieval itself runs two parallel searches, a BM25 lexical search over the inverted index using the augmented query, and a k-nearest neighbor search over the stored dense vectors using the original rewritten query. The results of both searches are merged using Reciprocal Rank Fusion. The fused set of chunks is then passed to a cross-encoder model, **ms-marco-MiniLM-L-6-v2**, which scores each chunk combined with the query to produce a final ranked list. The top-k ranked chunks from this list are passed to the generation subsystem.

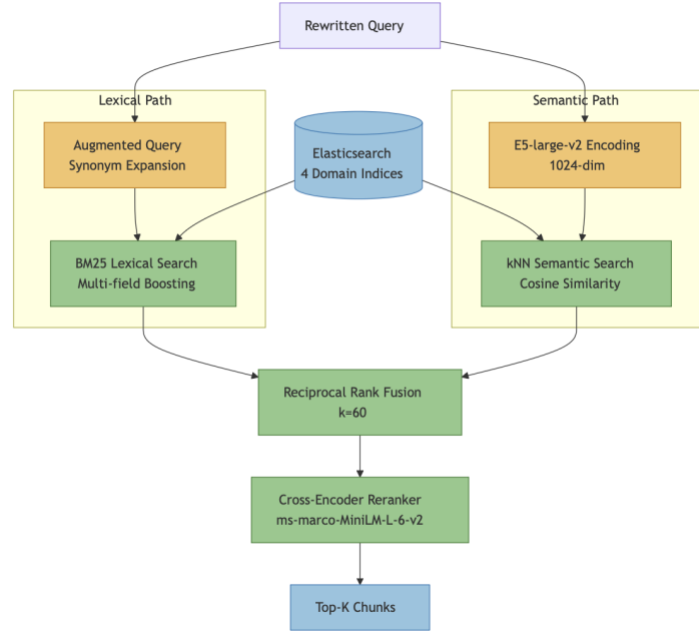


Figure 3: Retrieval Pipeline

3.1.3 Generation Subsystem

The generation subsystem receives the top-k ranked document chunks from the reranker and generates a grounded answer using **GPT-4o** through **Azure OpenAI**. The model is given a structured prompt that includes the retrieved chunks as context, the original user query, and explicit rules requiring the answer to be drawn only from the provided context. This grounding constraint is important to the system's reliability: IIT-specific information such as current tuition rates, staff contact details, and semester dates cannot be trusted from a language model's parametric knowledge, which is both outdated and inconsistent. If the retrieved context does not contain sufficient information to answer the query, the model is instructed to say so rather than speculate. A fallback path to LLaMA 3.1-8B through Groq is maintained for cases where the primary generation service is unavailable.

The completed answer is delivered through one of two interfaces. A Streamlit web application provides direct access for end users. A REST API built with FastAPI exposes the full pipeline to external clients through a single endpoint that accepts a query, an optional clarification context object from a prior turn, and a conversation history, and returns the generated answer alongside any new clarification state. The API is fully stateless: all session context is carried in the request and response objects rather than stored server-side, allowing any frontend to integrate the pipeline without session management.

3.2 Key Components

Component	Method Used	Why
Chunking	Topic Based + Sentence Based Chunking (Unstructured Data)	Keeps related content grouped by topic and splits at natural sentence boundaries, so retrieved chunks are complete and meaningful.
	Row Based (Structured Data)	Each record in structured data is already a self-contained unit, so no chunking is needed. One record map directly to one Elasticsearch document.
Indexing	Calendar Index	Stores structured event data with date fields enabling date-range filtering for precise calendar queries.
	Contacts Index	Stores staff directory records with keyword fields for exact department and category filtering alongside semantic vectors for natural language queries.
	Tuition Index	Stores fee records with school, level, and fee kind fields enabling pre-query filtering to prevent results from wrong programs reaching the LLM.
	Documents Index	Stores chunked policy documents with provenance metadata, document type, topic, page range enabling source attribution in generated responses.
Routing	Semantic Routing	a. Keyword based routing failed on natural language queries. b. LLM-based routing was slow and cost problems. c. Embedding similarity against domain prototype questions is fast, deterministic, and generalizes well to unseen phrasing.
Clarification	Slot filling - Calendar	Queries without a term are ambiguous across semesters, so slot filling asks for the term before retrieving.
	Slot filling - Contacts	Queries without a department or office are too broad across 1,552 records, so slot filling asks which department the user is looking for.

	Slot filling - Tuition	Queries without a school and level are ambiguous across 15 combinations, so slot filling asks which school and student level before retrieving.
	Slot filling - Documents	Queries without a policy topic are too vague across 599 chunks, so slot filling asks which policy area the user is asking about.
Retrieval	Query Rewriting - Semantic Search	Conversational phrasing contains no terms retrievable from the index. Rewriting converts casual language into formal academic vocabulary that matches indexed content. It is for the Semantic Search path.
	Query Augmentation - Lexical Search	BM25 relies on exact keyword matching, so informal or abbreviated queries miss relevant documents. Domain specific synonym expansion adds related terms to improve recall on the lexical search path.
	Lexical Search - BM25	Exact keyword matching retrieves documents with precise term overlap, particularly effective for structured fields like fee names, event names, and department names where exact terminology matters.
	Semantic Search - kNN	Captures meaning beyond exact keywords, handling paraphrasing and informal phrasing that BM25 misses. Uses the rewritten query embedded with E5-large-v2 to find contextually similar documents
	RRF - Reciprocal Rank Fusion	Combining BM25 and semantic scores directly is sensitive to score scale differences between the two systems. RRF merges ranked lists using position rather than score magnitude, producing a stable and robust combined ranking.
Reranker	Reranker - Cross Encoder	Bi-encoder similarity scores query and document independently, which is less accurate. A cross-encoder scores both together, producing more precise relevance estimates on the small candidate set from RRF
LLM	GPT 4o-mini	Generates grounded answers from retrieved context with explicit constraints to prevent the model from

		drawing on parametric knowledge, which is outdated and inconsistent for IIT-specific information like tuition rates and deadlines.
API	Rest API	FastAPI provides built-in request validation and automatic API documentation, which simplifies integration with the external team.
Hosting ES	ThetaCloud	Provided GPU enabled persistent storage without requiring Docker, allowing the E5-large-v2 embedding model to run and the pipeline.
Frontend	Streamlit	Native Python integration allowed direct connection to the backend pipeline without a separate server.

Table 2: Key Components

3.3 Design Decisions

3.3.1 Chunking Strategies

Two chunking strategies were applied, each matched to the structure of its source data. For unstructured documents, PDFs and HTML policy pages, a Topic-Based and Sentence-Based hybrid approach was used. Documents were first divided along topic boundaries such as section headings, then were further split at sentence boundaries to produce chunks of consistent size. This preserved topical coherence while ensuring each chunk ended at an end point rather than mid-sentence.

Four strategies were evaluated before this decision. Topic and Sentence splits first at topic boundaries then at sentence boundaries, keeping chunks coherent and complete with no overlap required. Topic and Fixed follows the same initial topic split but cuts remaining text into fixed length windows; this preserved topic structure but frequently truncated chunks mid sentence, requiring overlap to compensate. Topic and Recursive subdivides recursively from paragraph to sentence to word until a target size is met, which handles irregular document lengths. Topic and Semantic used an embedding model to detect semantic shift points within each topic segment, producing the highest quality chunk boundaries, but required a full inference pass over the document corpus at index time, ruling it out on the available hardware. Topic and Sentence was

selected because it matched Topic and Semantic in retrieval quality for this data, carried no inference overhead, and required no overlap to maintain coherence between adjacent chunks.

For structured data, calendar events, tuition records, and contact entries, each source row was indexed directly as a single document. No further splitting was needed since each record already represented one meaningful unit.

Chunk size calibration was the most persistent problem. Initially produced chunks had many short chunks, under 50 tokens, when PDF topic sections were brief; these lacked enough context to be useful in retrieval. Dense policy sections at the other extreme produced chunks spanning several hundred tokens, where the correct answer was buried alongside unrelated policy text and occasionally led to incomplete responses from the model. A minimum token threshold was introduced to merge short adjacent chunks, and sentence-boundary preservation with consistent target sizes was added at the end.

3.3.2 Schema Design and Index Mappings

Prior to data ingestion, domain specific schemas were developed for academic calendar, tuition, contacts, and documents. Each domain was assigned its own Elasticsearch index. The academic calendar, tuition, and contacts domains contain structured data, while the documents domain contains unstructured chunked text. Consolidating different kinds of data into a single index would have collapsed meaningful structural differences and reduced retrieval precision [1].

A consistent dual field design pattern was applied across all four indices, each record stores both raw structured fields and a semantic vector field of 1024 dimensions dense vector that was produced by the **E5-large-v2 model** [3] using cosine similarity. For the calendar, contacts, and tuition indices, the semantic text field is constructed during ingestion time from the structured fields and then embedded; the documents index embeds its content field directly, since that content is already natural prose.

Field types across all indices follow a convention i.e., descriptive. The content fields use Elasticsearch's text type, while identifier and categorical fields such as school, level, academic year, fee kind, email, and phone use the keyword type for exact-match filtering. Pre-filtering on keyword fields before semantic scoring is particularly important in the tuition index, where each record is bound to a specific school, level, and fee type; returning tuition records from an incorrect programme is problematic as the language model cannot verify the correctness independently [2].

The contacts mapping includes fields for name, department, category, description, phone, fax, email, building, address, city, state, and zip. The name and department fields are indexed as both text and keyword sub fields, this helps with having both full text matching and exact filtering. The tuition mapping additionally derives a fee kind field from fee name during ingestion, normalising fee categories for consistent filtering. The documents index schema has fields such as chunk id, document type, document name, topic, and token count to enable source attribution in generated responses.

Several schema design issues were found during development and addressed iteratively. The original calendar mapping used a single event date field, which could not represent multiple day events; this was replaced with separate start date and end date fields. The calendar's semantic text was initially generated from a simple template, but this produced poor representations where precision of phrasing was essential; the final design replaced the template with a manually authored lookup table covering all 118 calendar events, each phrased to explicitly state consequences and surrounding context. In the tuition index, an early strict filter on fee kind caused queries to return no results; this was resolved by adding a classification step that applies the filter only for broad per credit tuition questions and bypasses it for queries mentioning continuation, mandatory, or all fees.

3.3.3 Indexing and Ingestion

Each of the domains was indexed through a dedicated script that loaded the source data, generated dense vector embeddings based on dedicated semantic text fields, and wrote documents into the corresponding index. For the calendar, contacts, and tuition domains, an additional semantic text field was built from the structured fields before embedding; the documents index embedded its content field directly. **E5-large-v2** was used as the embedding model, with a 1024-dimensional output vector, encoded with a "passage: " prefix as required by the model's encoding scheme and L2 normalisation before storage. The model instance was loaded once at startup and was reused across both indexing and operations at query time. Having dedicated ingestion scripts per domain isolated the indexing logic and allowed any of the indices to be re-indexed without affecting the other indices.

One change was made to the document's ingestion pipeline during development: the original design loaded chunks from a directory of individual document JSON files, which proved brittle to maintain as the document set grew. This was consolidated into a single pre-processed file, simplifying ingestion and reducing the chance of errors during indexing.

3.3.4 Out of Domain

User queries first pass through a pre-router gate designed to intercept messages that cannot be meaningfully answered by the university knowledge base. The gate handles short conversational openers such as greetings and acknowledgements, out-of-domain queries or unrelated queries, and then a few emotional queries that are acknowledged rather than retrieved. Longer inputs pass directly to the semantic router.

The implementation combines an exact-match set for common short phrases with compiled regular expression patterns for broader categories such as national politics or unrelated institutions. Out of domain detection operates at two layers. The pre-router gate handles unambiguous cases at zero cost, incurring any embedding or LLM cost. This optimisation is practically significant, as a non-trivial fraction of user queries in deployment are greetings or off-topic, and routing them through the full pipeline would waste computation and risk returning irrelevant responses to inputs that carry no informational intent [4].

3.3.5 Semantic Query Routing

The router determines which of the four domain pipelines should execute a given query. The system uses a prototype-based semantic similarity approach that requires no labelled data. A domain specific set of representative questions were manually curated for each domain, covering a wide range of phrasings and intent of users.

Prototype embeddings are precomputed once at startup using E5-large-v2 with the "query: " prefix and cached in memory. At query time, the input is embedded with the same prefix and compared against every prototype via cosine similarity. The router takes the top-K scoring prototypes across all domains, then assigns each domain the maximum score it achieved within that set. A domain is selected if its score meets a confidence threshold; queries that fall below the threshold the domain is considered as out of domain and returns a fallback response without entering the retrieval pipeline. For queries under three words, the threshold was changed from 0.50 to 0.68, since short inputs tend to produce lower confidence matches that would otherwise be misrouted.

Multi-domain routing is triggered when a second domain's score falls within 75% of the top score, in which case both pipelines execute in parallel and their results are merged. This handled queries that genuinely straddle two topics, such as questions about tuition deadlines that also touched the academic calendar.

The prototype approach was preferred over a fine-tuned classifier as it requires no labelled data and is easy to extend adding coverage for a new intent means appending prototype questions rather than retraining. The main limitation encountered was sensitivity to phrasing gaps in the prototype sets; queries using unexpected terminology occasionally fell below the confidence threshold even when they were clearly in-domain. This was fixed by iteratively expanding the prototype sets after reviewing failure cases during evaluation.

3.3.6 Query Rewriting

Following routing, the raw query is reformulated into a cleaner, more meaningful form before the search pipeline. Conversational queries frequently contain pronouns, implicit references, or informal phrasing that both lexical and semantic retrievers handle poorly. A query such as "what about grad students?" is semantically underspecified without its preceding turn, and "is it expensive?" contains no terms retrievable from a university knowledge base.

The rewriting step uses **LLaMA 3.1-8B** accessed via Groq API to produce a reformulated query. When a prior user turn is available, it is passed as a context hint for co-reference resolution, allowing short follow-ups to be expanded with entities from the preceding question. The rewritten query propagates to the augmentation and retrieval stages, while the original query is retained separately for use at the reranking step. This separation ensures that rewriting improves recall without distorting the relevance signal at the reranking stage [5].

Two cases where rewriting caused retrieval regressions were identified during development and bypassed. First, tuition comparison queries containing terms like "compare" or "vs" were found to drift toward unrelated programme text after rewriting, so these are passed through unchanged.

Second, document only queries that are already specific were found to be paraphrased into vague forms that hurt retrieval, so rewriting is skipped when the query is self contained and no prior context is needed. A secondary rewriting routine handles clarification responses by merging the original underspecified query with the user's clarification answer into a single reformulated query, which then re-enters the pipeline as if it had been asked directly.

3.3.7 Slot-Filling and Clarification

Certain domains require specific parameters before retrieval can be performed meaningfully. A tuition query without a school or student level would return fee records spanning all programmes, and a calendar query without a term would combine events across multiple semesters. Each domain pipeline therefore includes a slot filling step that checks the rewritten query for required parameters before retrieval begins.

The validation logic is built from patterns compiled at first use from the indexed data. For the tuition domain, two pattern layers are used. The first, called the anchor layer, confirms that tuition intent is present using school names, fee names, levels, years, and structural vocabulary such as "per credit" or "billing". The second layer checks for specificity, whether the query names an actual school level, fee, or year rather than just generic cost vocabulary. Clarification is only triggered when the anchor matches but the specific layer does not, meaning the user is asking about tuition but has not identified which programme or level. For the calendar domain, named holidays and specific events accompanied by an explicit term or date pass through directly; queries containing only weak vocabulary such as "deadline" or "start" without a semester or year are flagged for clarification. For contacts, department names are expanded at initialisation so that informal queries using names like "computer science" or "Kent Law" resolve against the full department name. A curated list of 17 common offices and schools are shown as options. For documents, queries matching topic or document-type keywords from a config file pass through; queries of five or more words that do not match generic filler also pass through to avoid over-clarifying long questions.

The calendar domain includes a second clarification gate inside the retrieval router itself, which runs after slot-filling. This gate first checks whether the query contains a specific date expression. If a full date, month and day, or month with a preposition is found, the router runs a date-range filter query directly against Elasticsearch to retrieve events that fall on or span that date, bypassing semantic search entirely. If no events are found on the given date, the router treats it as a reference point rather than an event and falls through to semantic search. Queries for a full holiday list are routed to a dedicated search function that returns all holiday and break events sorted by date rather than the top-K semantic results. If the query mentions an event keyword such as "finals", "deadline", or "registration" but contains no semester, year, or month, the router re-asks for the semester even if the earlier slot-filling gate had passed it through.

If a required slot is absent at either gate, the pipeline emits a structured clarification request with a prompt and an option list. On the user's next turn, the reply is classified as a direct answer, a cancellation, or a new topic. If a valid answer is received, it is merged with the original query and re-enters the pipeline. All four validators originally used a simpler rule that passed any query of three or more words straight to retrieval. This was replaced after multi-word but underspecified queries such as "what are the fees" or "what is the calendar" consistently returned poor results.

3.3.8 Query Augmentation

To improve lexical recall, the rewritten query goes through synonym expansion before the BM25 search. The augmentation step appends domain specific synonyms, producing an expanded query used only for lexical retrieval.

Each domain has its own synonym dictionary mapping key terms to one or two closest equivalents. When a key term is found in the query, its synonyms are appended up to a cap of three new terms per query. Synonyms that duplicate tokens already present in the query are skipped. The dictionaries are deliberately narrow as that kind of expansion was found to add BM25 noise without improving recall. For the documents domain only, the expanded query is also passed to the reranker rather than the original query, so the cross-encoder scores candidates against the same enriched terms used in lexical retrieval.

Synonym expansion is intentionally not applied to the semantic search, which retains the original rewritten query. Semantic retrieval with E5-large-v2 already captures paraphrase similarities through its embedding space, and injecting synonym noise into the query vector would distort the representation rather than enrich it [3]. Restricting augmentation to the lexical search therefore allows BM25 to benefit from broader term coverage while keeping the semantic signal faithful to the user's original intent.

3.3.9 Hybrid Retrieval

The retrieval layer is the core of this system and combines two retrieval methods to maximise both precision and recall. For each query, two parallel searches are conducted against the domain specific index: a lexical search and a semantic search, whose results are fused before reranking.

Lexical search uses BM25, Elasticsearch's default probabilistic retrieval model [6]. Before the search, common stopwords are stripped from the query to reduce noise in term frequency scoring. Field level boosting is applied per domain to concentrate BM25 scoring on the most discriminative fields, the calendar index heavily weights event name and adds a phrase match boost to reward exact event name matches; the documents index weights topic highest; the contacts index weights name and department; and the tuition index weights chunk text and fee name. Semantic search uses cosine similarity over the dense vector representations, scored in Elasticsearch via a script score formula that adds 1.0 to the raw cosine value to keep all scores positive.

For the tuition domain, both searches are additionally constrained by structured filters extracted from the query before retrieval: school name, student level, and academic year are identified through alias mapping and regex and applied as Elasticsearch filter clauses. If the combined filter returns no results, a fallback chain retries progressively: first without the level filter, then without the fee kind filter, and finally unfiltered. When a school is specified but no level is mentioned, the tuition pipeline runs separate searches per level and merges the results so the answer generation stage receives fee records for all student levels.

The two ranked lists are combined using Reciprocal Rank Fusion with $k = 60$, which scores each document as the sum of $1/(60 + \text{rank})$ from each list [7]. RRF was chosen over a weighted score combination after an earlier implementation that linearly blended BM25 and cosine scores using fixed weights of 0.6 and 0.4 proved unreliable; the two scoring scales are not comparable across queries, making the blend sensitive to document length and index statistics in ways that were difficult to control. RRF sidesteps this by operating on rank positions rather than raw scores.

3.3.10 Cross-Encoder Reranking

The fused set produced by RRF is then passed through a cross-encoder reranker before the generation stage. The model used is **ms-marco-MiniLM-L-6-v2**, a lightweight cross-encoder fine-tuned on the MS MARCO passage ranking benchmark [8].

During retrieval, bi-encoders embed the query and documents independently and compare them using vector similarity. While this is efficient at scale, it means the model never directly attends to the interaction between query and document terms. A cross-encoder, in contrast, takes the query and a candidate document as a combined input and scores them as one, allowing it to capture relevance signals that bi-encoders miss [5]. The trade-off is computational cost; cross-encoders are too slow to apply across the full index. In this pipeline, the reranker is therefore applied only to the top chunks returned by hybrid retrieval, making it tractable while improving the quality of results passed to the generation stage.

After scoring, a topic-match boost is added to each chunk's cross-encoder score proportional to the fraction of content words from the query that appear in the chunk's topic field, with a maximum boost of 8.0. This was introduced after observing that policy document chunks with highly relevant topic labels but generic opening sentences were being ranked below less relevant chunks that happened to open with more query-matching language. The final reranked set is capped at 5 results for calendar, contacts, and tuition, 7 for documents, and 10 for tuition queries where level diversification is active.

3.3.11 Answer Generation

The top-k ranked passages from the reranker are assembled into a structured prompt that is passed to GPT-4o through the Azure OpenAI API for answer generation with LLaMA 3.1-8B via Groq as a fallback if the primary call fails. The prompt presents the retrieved context alongside the original user query and includes explicit rules requiring the model to ground its answer in the provided context rather than external knowledge. This constraint is critical for factual accuracy.

The system prompt contains explicit behavioural constraints. The model is instructed to ground every claim in the provided context and to respond with "I don't have that information available." if the answer is absent, rather than drawing on training knowledge. Exact figures, dollar amounts, dates, GPA thresholds, credit hours, and must be stated verbatim as they appear in the context. The model is also instructed not to open with conversational filler such as "Great question!" or "Certainly!", and not to close with pleasantries. A specific instruction handles multi-level tuition queries: when context contains fee records for multiple student levels and the user has not specified one, the model must list rates for all levels rather than selecting one. Conversation history is

restricted to pronoun and short-reference resolution and must not bleed prior answers into the current response.

GPT-4o was selected for its ability to follow instructions precisely and generate coherent answers from multiple chunks. The top chunks frequently cover the same topic from different source documents, and the model is expected to produce one consolidated answer rather than repeat content. The generation stage also handles output formatting to ensure responses remain clean across different frontend environments.

3.3.12 API Layer

The complete pipeline is exposed as a stateless REST API implemented in FastAPI. The primary endpoint accepts a user query, an optional chat history for multi-turn context, and clarification state from a previous turn if applicable. It returns the generated answer, source urls, and a flag indicating whether the reply is itself a clarification request. All session state is passed with each request rather than stored server side, which keeps the API simple to scale and allows any frontend to connect.

One integration issue arose during development. The external frontend always sent the clarification state as null rather than echoing it back, which caused the clarification flow to loop. A workaround was added to detect a single entity and route them directly to the right pipeline without requiring the clarification state to be present.

FastAPI was chosen over alternatives such as Flask for its built-in request validation and automatic API documentation, which simplified integration with the external team [9].

3.3.13 Frontend

The user interface was built using Streamlit, chosen for its rapid prototyping capability and native Python integration with the backend pipeline. The application has two pages: a landing page describing the system's scope and limitations, and a chat page for querying. Streamlit's chat input and chat message components were used to build the conversational interface. Session state-maintained conversation history and clarification context across turns, and each request to the FastAPI backend carried the user query, chat history, and clarification state as a combined payload. Source URLs returned by the API were displayed beneath each answer, linking users directly to the official IIT page the response was drawn from.

Streamlit's session state is local to a single browser tab and does not persist across page reloads. During cross group evaluation, both frontends were connected to both backends simultaneously. This caused each chatbot to receive the other pipeline's clarification state, which risked altering its response and breaking the conversational flow. Careful alignment of request and response schemas was required to ensure each pipeline only consumed its own clarification state.

4. Development Process

Week	Goal	What was done	Issues
1	Understanding the project and researching through ideas and methods.	Reviewed literature on RAG systems and evaluated different retrieval strategies.	-
2	Identifying data sources and more research on Elasticsearch	Data sources were collected and consolidated into an excel sheet.	The extent of how much data and data sources were required to collect was an issue.
3	Build a basic prototype to understand how Lexical search works and also how Elasticsearch works.	Set up Elasticsearch, indexed academic calendar data, tuition and fees data, implemented basic BM25 lexical search, tested query retrieval on calendar domain to understand how Elasticsearch scoring and indexing works	Lexical search alone was insufficient, keyword mismatches returned incorrect results.
4	Data collection and preprocessing setup	HTML scraping was implemented using BeautifulSoup and PDF parsing using pdfplumber. Extracted content was manually inspected against source documents to validate chunk boundaries. The initial embedding model was selected as all-MiniLM-L6-v2 and evaluation metrics as precision, hallucination rate, and completeness.	JSON was selected over plain text as the storage format for extracted data. PDF documents lacked explicit semantic markers, and layout-based extraction occasionally missed section boundaries, requiring manual review of affected chunks.
5	Implement lexical search on structured data and evaluate its effectiveness	Indexed Tuition and Calendar data into Elasticsearch, designed BM25 lexical queries with field boosting and phrase matching, ran test queries against the index	Lexical search alone proved to be insufficient for structured data due to keyword mismatches, informal phrasing, and synonym gaps, causing poor retrieval.

6	Research and selected optimal chunking strategy for unstructured documents	Evaluated four hybrid chunking strategies (Topic+Sentence, Topic+Fixed, Topic+Recursive, Topic+Semantic); selected Topic+Sentence as it preserved both structure and meaning with low complexity and no overlap was needed	Topic+Semantic offered higher accuracy but was rejected due to high computational complexity
7	Implement hybrid search and enriching indices with semantic vectors	Built hybrid retrieval pipeline using BM25 + all-MiniLM-L6-v2 (384-dim embeddings). Designed calendar index mapping along with fusing scores using weighted sum ($0.6 \times \text{BM25} + 0.4 \times \text{cosine}$)	Weighted score fusion was sensitive to score scale differences between BM25 and cosine, making combined rankings unstable.
8	Extend hybrid search to documents and contacts, evaluate routing strategies, and test contacts retrieval.	Unstructured documents and contacts were indexed using all-MiniLM-L6-v2 with RRF fusion combining BM25 and semantic search. Four routing strategies were evaluated: deterministic keyword-based, embedding-based, score-based, and zero-shot LLM classification. Deterministic routing failed on natural language and compound queries; zero-shot LLM routing struggled with missing intent and context. Contacts retrieval was tested across all three approaches and hybrid outperformed both lexical and semantic on every category. The initial chatbot was also evaluated on 100 test questions.	No routing strategy performed reliably on its own. Session state was not maintained between turns, and LLM responses were inconsistent across repeated queries.

9	Upgrade chunking, implement semantic router, query augmentation, slot filling, cross-encoder reranker, and evaluate full pipeline	Chunking was improved with sentence boundary preservation and consistent target chunk sizes. The LLM-based router was replaced with semantic routing using embedding similarity against domain prototype question sets, with multi-domain routing added for queries spanning two domains. Query augmentation was implemented via domain-specific synonym expansion on the lexical path, and LLM-based query rewriting was added to convert conversational phrasing into formal retrieval vocabulary. Slot filling was added across all four domains to detect missing parameters such as term, school, department, and policy topic. Cross-encoder reranking using ms-marco-MiniLM-L-6-v2 was added after RRF fusion. The full pipeline was evaluated on 100 questions.	Documents retrieval significantly weaker than other domains; partial results still high due to multi-domain queries
10	Integrate all four indices into a single unified pipeline and improve the full system	Combined all domain indices into one chatbot; upgraded embedding model to E5-large-v2 (1024-dim); improved routing, retrieval, and LLM selection.	ThetaCloud deployment was unstable. Retrieval Quality issues.
11	Comprehensive evaluation of the full pipeline across single-turn, multi-turn, and blind test scenarios	The blind test pass rate of 57.1% was substantially lower than the other two test types, indicating the pipeline was less robust on unseen query phrasings. Performance was inconsistent across	Blind test pass rate significantly lower at 57.1%; inconsistency across some categories; Tuition and Transfer Credit weakest performers

		categories, with Tuition and Transfer Credit as the weakest.	
12	Joint evaluation and frontend design with Group 6 covering both pipelines	Designed comprehensive 88-question evaluation plan covering calendar, contact, policy, registration, transcripts, tuition, financial aid, and multi-topic queries; defined evaluation metrics including overall performance, performance by category and level, failure rate analysis, stress questions, clarification questions, query session memory, and retrieval metrics; compared both pipelines on pipeline quality scores	-

Table 3: Project Timeline

5. Evaluation

5.1 Within-Group Evaluation

The system was evaluated iteratively across multiple rounds of testing, covering four distinct dimensions: intent routing accuracy, single-turn query performance, multi-turn session behavior, and blind testing on unseen queries. Each round was designed to surface a different aspect of system behavior, from the correctness of domain classification to the robustness of clarification dialogs under realistic conversational conditions. All evaluations were conducted internally by the project team.

5.1.1 Router Accuracy

The intent router was evaluated on 25 queries spanning all four domains. Each query was assessed as either correct (routed to the exact expected domain) or partial (routed to the correct domain but with an additional unnecessary domain included).

Result	Count	Percentage
Correct	17	68%
Partial	8	32%

Table 4: Router evaluation results

The router achieved correct classification on 68% of queries. The partial cases primarily involved queries being routed to an additional domain alongside the correct one, most commonly Documents being included alongside Contacts or Calendar queries. No queries were routed entirely to the wrong domain, indicating that the GPT-4o classifier reliably identifies the primary domain but occasionally over-triggers on secondary domains.

5.1.2 Single-Turn Query Evaluation

A set of 60 questions was evaluated across five domains including an out-of-scope category to test system boundaries. Each response was assessed as either pass or fail based on factual correctness and completeness against expected answers derived from official IIT sources.

Domain	Questions	Pass	Fail	Pass rate
Calendar	15	11	4	73%
Tuition	15	11	4	73%
Contacts	10	8	2	80%
Documents	15	14	1	93%
Out of scope	5	4	1	80%
Total	60	48	12	80%

Table 5: Direct question evaluation results

The Documents domain achieved the highest pass rate at 93%, reflecting the effectiveness of the hybrid BM25 and kNN retrieval pipeline for policy and procedure questions. Calendar and Tuition both achieved 73%, with failures primarily occurring on underspecified queries where the system answered without first requesting clarification. The out-of-scope category achieved 80%, indicating the system correctly declined to answer or redirected the user in most cases.

5.1.3 Multi-Turn Session Evaluation

Multi-turn behavior was evaluated across 30 scenarios testing three capabilities: clarification dialog, follow-up question handling, and edge case robustness.

Capability	Score	Pass rate
Clarification	8.5/10	85%
Follow-ups	3.5/10	35%
Edge cases	7/10	70%
Overall	19/30	63%

Table 6: Multi-turn question evaluation results

Clarification dialog performed strongest at 85%, reflecting the slot-filling and query expansion logic working as intended for underspecified queries. Edge case handling achieved 70%, covering scenarios such as out-of-domain follow-ups and mid-session topic switches. Follow-up question handling remained the weakest area at 35%, reflecting the difficulty of maintaining conversational context accurately when the topic shifts between domains across turns.

5.1.4 Blind Test

A blind test was conducted on 42 turns covering queries the system had not been specifically tuned against. Responses were assessed as pass, partial, or fail.

Topic	Pass	Partial	Fail	Total	Pass rate
Calendar	6	0	3	9	66.7%
Student Policies	8	2	3	13	61.5%
Tuition Fees	6	1	2	9	66.7%
Contacts	2	1	4	7	28.6%
Edge Case	2	0	0	2	100%
Out of Scope	0	0	2	2	0%
Total	24	4	14	42	57.1%

Table 7: Blind test evaluation results

The blind test surfaced two consistent weaknesses. Contacts performed at 28.6%, primarily on queries requiring specific directory lookups or the query phrasing did not match indexed fields. Out-of-scope queries achieved 0%, indicating the system attempted to answer questions it should have declined, a known limitation of the current domain boundary logic. Calendar, Tuition, and Student Policies performed consistently in the 62-67% range on unseen queries, which is acceptable given the diversity and difficulty of the blind test set compared to the structured evaluation.

Taken together, the four evaluation dimensions reveal a consistent pattern. The system performs strongly on well-formed, domain-specific queries where retrieval targets are clear, as reflected in the 80% overall pass rate in single-turn evaluation and the 93% pass rate for Documents specifically. However, the blind test results present a more realistic picture of real-world performance: Documents-heavy categories such as Student Policies dropped to 61.5% on unseen conversational queries, indicating that the structured single-turn evaluation likely overstates performance on ambiguous or naturally phrased questions. These findings collectively point to retrieval robustness on conversational queries and out-of-scope boundary enforcement as the primary areas for future improvement.

5.2 Cross-Group Comparison

The evaluation was conducted across multiple systems built in the same course setting, all answering the same set of questions drawn from official IIT university sources. Two broad retrieval approaches were represented: a pure Retrieval-Augmented Generation approach and the Hybrid Elasticsearch with RAG approach developed by this project. A NotebookLM baseline was also included as an AI reference point to contextualize both systems against a general-purpose document-grounded assistant.

The table below summarizes the qualitative differences between the two approaches across three dimensions.

System	Strengths	Weaknesses	When it works best
RAG	Strong on broad policy questions where contextual coverage matters more than precision. Simpler pipeline with fewer moving parts	Prone to retrieving semantically similar but factually off-target chunks. No structured filtering for domain-specific fields like fee amounts or calendar dates	Open-ended policy and procedure questions where the answer spans multiple paragraphs of a single document
ES	High precision on fact-specific queries involving exact values, dates, and contact details. Domain-specific slot-filling and query expansion improve retrieval quality across all four domains. Cross-encoder reranking ensures the most relevant chunks reach the LLM	More complex pipeline with domain-specific components that require maintenance. Performance drops on ambiguous multi-topic queries that span several domains simultaneously	Structured queries with specific entities such as school names, fee types, semester terms, or office contacts. Also effective for multi-turn clarification dialogs

Table 8: Qualitative Comparison of Retrieval Approaches: RAG vs Hybrid ES+RAG

Each system response was evaluated by the project team using a five-point scoring rubric:

- 5 (Perfect) indicates a correct, complete, and well-formatted answer
- 4 (Good) indicates a correct answer missing a minor detail
- 3 (Partial) indicates a partially correct or incomplete response
- 2 (Poor) indicates a mostly wrong or too vague answer
- 1 (Fail) indicates a completely wrong response or a refusal to answer.

All scores were assigned independently by team members based on comparison against expected answers derived from official IIT sources.

Metric	RAG	Hybrid ES + RAG
Mean Square	4.15	4.28
% Perfect (5)	54.4%	66.3%
% Low Quality (≤ 2)	11.1%	11.2%
SD	1.16	1.18

Table 9: Quantitative Comparison between RAG and Hybrid ES+RAG

System	Mean Score	Median	SD	Perfect (5)	% Perfect	Low Quality	% Low Quality
Linear	4.06	5	1.31	50	55.6%	25	13.3%
Self Query	3.69	4	1.40	38	43.2%	36	23.9%
Traffic Cop	4.15	5	1.16	49	54.4%	22	11.1%
ES+RAG	4.28	5	1.18	59	66.3%	10	11.2%
NotebookLM	4.61	5	1.03	74	83.1%	9	6.7%

Table 10: Overall Mean Score Comparison Across All Systems

Category	N	Mean Score	SD	Perfect (5)	% Perfect	Low Quality	% Low Quality
Calendar	14	4.57	1.16	12	85.7	1	7.1
Contacts	9	4.67	1.00	8	88.9	1	11.1
Financial Aid	8	4.00	0.76	2	25.0	0	0
Multi-topic	9	3.89	1.69	6	66.7	3	33.3
Policy	12	4.00	1.28	6	50.0	1	8.3
Registration	8	3.88	1.55	4	50.0	2	25.0
Transcripts	9	4.56	0.88	7	77.8	0	0
Transfer Credit	9	4.44	0.88	6	66.7	0	0
Tuition	11	4.36	1.21	8	72.7	2	18.2
Overall	89	4.28	1.18	59	66.3	10	11.2

Table 11: Performance by Category - Hybrid (ES+RAG)

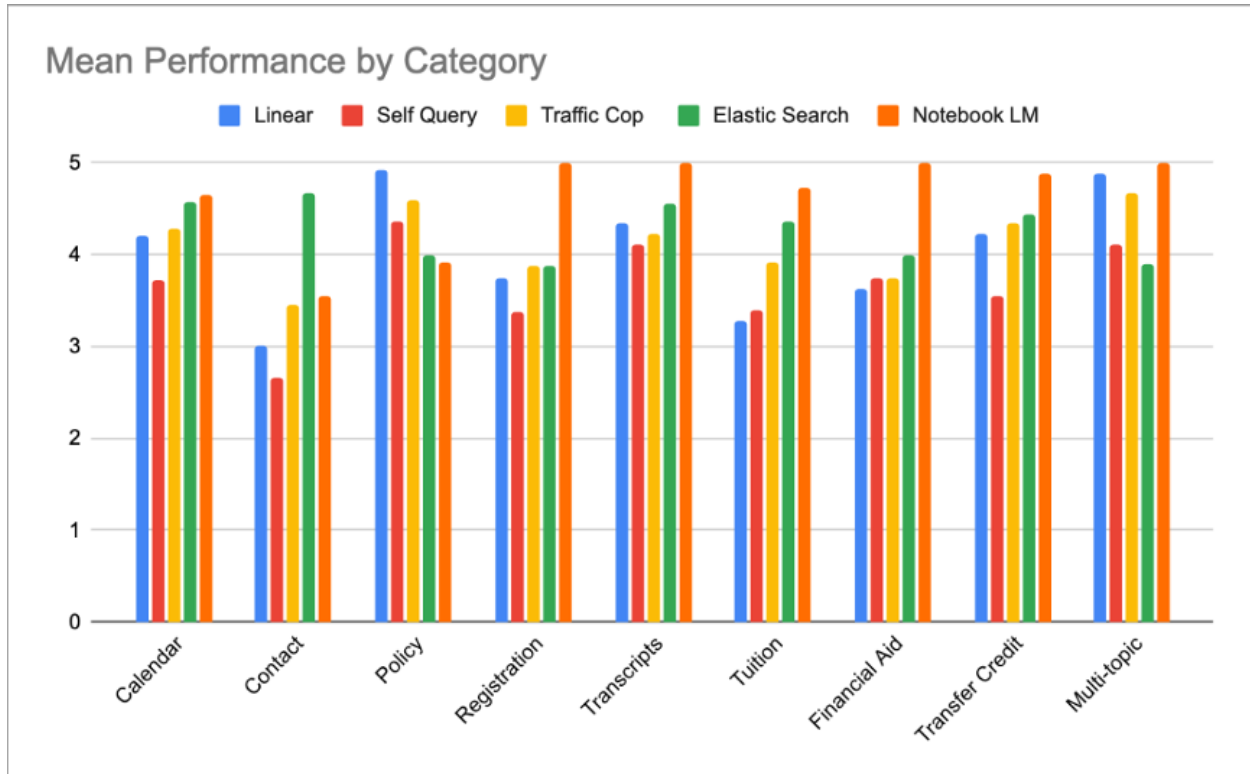


Figure 4: Mean Performance by Category - All Systems

Difficulty	Mean Score	SD
Easy	4.54	1.18
Medium	4.44	0.98
Hard	3.90	1.45

Table 12: Performance by Difficulty - Hybrid (ES+RAG)

Mean Performance by Difficulty Level

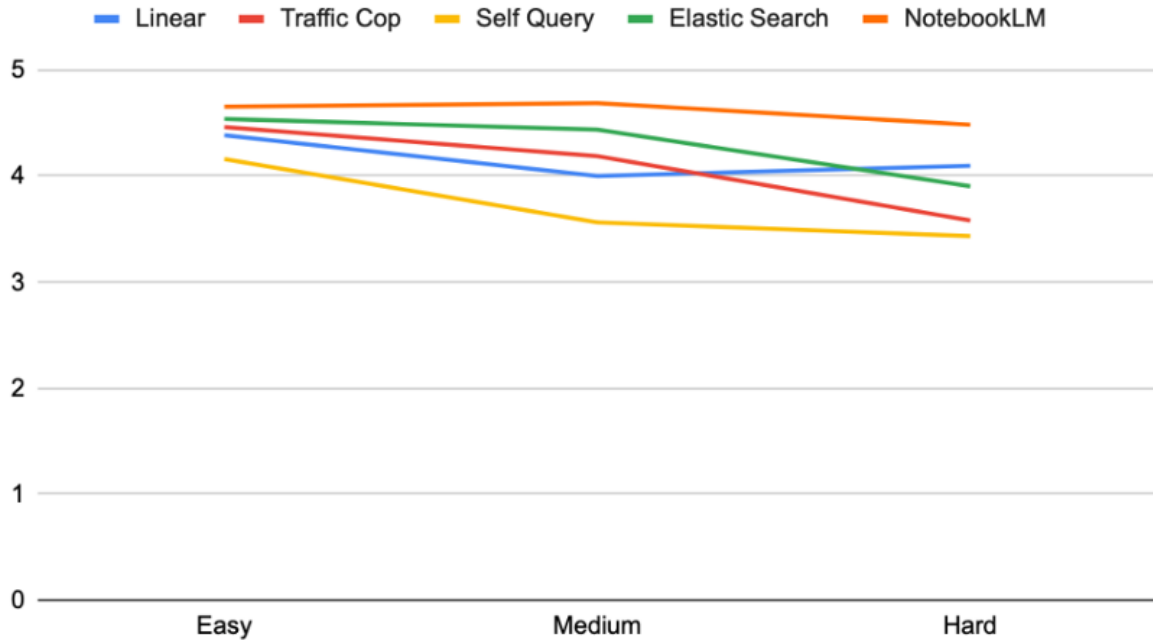


Figure 5: Mean Performance by Difficulty Level - All Systems

Across both difficulty levels and question categories, the Hybrid ES+RAG system demonstrated consistently strong performance, particularly in domains requiring precise fact retrieval such as contacts, transcripts, and calendar dates, achieving perfect scores on 85-89% of questions in those categories. Categories involving broader contextual reasoning such as financial aid and multi-topic queries showed greater variability, reflected in higher standard deviations and lower perfect score rates. This pattern is consistent with the design of the system: structured retrieval with slot-filling excels when queries have clear entity targets, while queries that require synthesizing information across multiple policy areas remain the harder challenge. Both systems performed below the NotebookLM baseline overall, which reflects the advantage of a general-purpose AI system with broader document grounding, though NotebookLM operates without the domain-specific structure and source constraints that make a university chatbot reliable and auditable.

5.3 Auto-RAG Comparison

NotebookLM was included in the evaluation as an Auto RAG baseline, a general-purpose document-grounded assistant that automatically retrieves and synthesizes information without domain-specific pipeline engineering. Comparing this baseline helps contextualize the tradeoffs between a manually engineered hybrid system and an automated approach.

Overall, NotebookLM achieved a higher mean score of 4.61 compared to the Hybrid ES+RAG system's 4.28, with a perfect answer rate of 83.1% against 66.3%. This gap is expected given that NotebookLM operates with a more powerful general purpose model and broader document grounding without the constraints of a structured retrieval pipeline.

However, the comparison surfaces some nuanced findings. The Hybrid ES+RAG system outperformed NotebookLM on the Contacts domain, achieving a mean score of 4.67 against NotebookLM's 3.56. This is the most structurally interesting result: Contacts queries benefit from indexed, queryable directory data with field-level matching, which a structured retrieval system handles more precisely than a general-purpose document reader. It suggests that for domains where data is highly structured and queries have specific entity targets, a purpose-built retrieval pipeline outperforms automated approaches.

NotebookLM performed notably stronger on Registration, Financial Aid, Transcripts, and Multi-topic queries, categories that require synthesizing information across multiple policy documents. This aligns with the known limitation of the Hybrid ES+RAG system where cross-document reasoning and multi-hop queries remain challenging.

The key distinction between the two approaches is not accuracy alone but explainability and auditability. The Hybrid ES+RAG system retrieves from a fixed, verified set of official IIT sources with traceable chunk-level provenance, whereas NotebookLM's retrieval process is opaque. For a university information system where answer accuracy and source accountability matter, this architectural difference is a meaningful practical advantage even where raw scores are lower.

5.4 Online Evaluation (User Study)

An online questioner was shared among faculty and students to test both the systems and provide a rating on their experience for the below metrics on a scale of 1-5; 1 being very poor and 5 being excellent.

The questioner was filled by 15 people.

Metric		Chatbot A	Chatbot B
Avg rating	Accuracy	3.8	3.5
	Relevance	3.57	3.78
	Clarity	3.5	3.85
	Satisfaction	3.5	4.07

Table 13: Online evaluation results

Which chatbot do you prefer overall?

15 responses

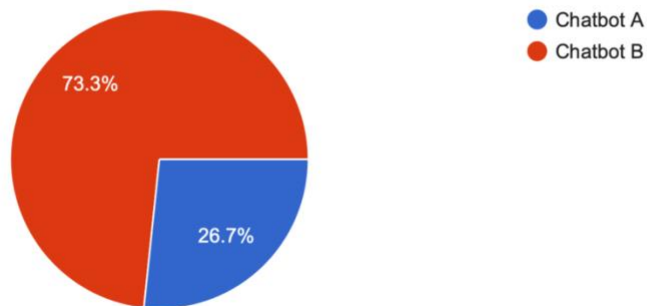


Figure 7: Preference Pie chart

5.4.1 User Comments

Here are some of the user comments that were provided

1. There are some leaks of the source URLs in the responses of Model A, the URLs could be presented in a clickable and concise fashion in the Model A responses (not the ones in sources). Model B responses are better didn't found any such leaks
2. The chatbot is surprisingly good, answers could be more specific and needs to consider input a little better.
3. Chatbot B is faster. It understands questions better and gives clear and correct answers
4. Chatbot B kept asking to specify my question. Chatbot A was very precise and accurate.
5. I asked basic question to the both chatbots " where is the business school located?" and chatbot A replied " The Stuart School of Business is part of the Illinois Institute of Technology. Specific location details for the business school are not found in the provided context. For further assistance, you may contact the Stuart School of Business directly through their directory " and chatbot B said " The Stuart School of Business is located at 565 West Adams Street, 4th Floor. "
6. Personally, I felt like Chatbot B gives more clear and understandable answers, covering all the possible scenarios I can expect a given situation. The answers given are more detailed, making one answer enough to knowing everything about what i asked.
7. Both the chat bots provided responses to my satisfaction. However chatbot B was more accurate and clear while responding.

6. Key Challenges

6.1 Technical Challenges

Purely lexical retrieval was insufficient for conversational queries. BM25 relies on term overlap, so queries phrased as natural questions such as "when does class start" or "who handles transcripts" failed to match index entries phrased as structured labels like "First Day of Instruction" or "Office of the Registrar". Domain-specific synonym expansion addressed a subset of these cases by appending known equivalents to the augmented query, but could not generalise to phrasings not anticipated during development. Dense retrieval with E5-large-v2 was required to bridge this gap. Dense retrieval alone degrades on entity heavy queries where exact matching is critical, specifically school names, fee categories, and specific dates, because embedding similarity compresses surface distinctions that carry meaning in this domain. A hybrid approach combining BM25 on the augmented query with kNN on the rewritten query, fused via Reciprocal Rank Fusion, was necessary to handle both query types reliably.

Designing the clarification flow for a stateless API required the backend to serialize domain context, detected missing slots, and clarification options into every response so the frontend could echo them back on the following turn. When upstream context was absent or malformed, the clarification loop could not resume correctly, requiring a fallback that re-derived routing intent from the raw query alone. Distinguishing a slot-fill answer from a topic switch, for example a user replying "graduate" after a level prompt versus asking an unrelated question, required separate detection logic to prevent context from bleeding across turns.

Queries referencing more than one domain simultaneously, such as "what is the tuition payment deadline for Mies", required routing to both the tuition and calendar indices. A multi-domain ratio threshold was set so that a secondary domain was included only when its top prototype similarity score fell within 75 percent of the primary domain score. Even with this filter, reranking across results from two indices occasionally surfaced low confidence secondary results that degraded response quality. A minimum reranker score threshold was applied as a second gate to suppress these before context was passed to the language model.

6.2 Deployment and Integration Challenges

Streamlit's session state does not persist across page reloads, so conversation history was lost whenever a user refreshed the browser. Since the backend was stateless by design, there was no server-side session to recover from; each reload started a fresh conversation with no memory of prior turns. Managing concurrent users on a shared Streamlit instance also required care, as session state is scoped per browser tab but the underlying FastAPI backend and Elasticsearch instance were shared across all active sessions.

Azure deployment introduced response latency beyond what local testing had indicated. Requests traversed three network hops: the Streamlit frontend, the FastAPI backend, and the Azure OpenAI endpoint. Each hop added overhead, and the combined latency pushed end-to-end response times to 3 to 6 seconds per query under normal load. The Azure OpenAI message token limit also constrained how much retrieved context could be included per request. When multiple long document chunks were retrieved, truncation was required to stay within the limit, which occasionally cut off relevant content before it reached the language model.

6.3 Evaluation Challenges

Selecting metrics that meaningfully capture both retrieval and generation quality required treating the two stages separately. Retrieval was evaluated using Precision@k, Recall@k, and Hit Rate@k against manually identified relevant chunks. Generation quality was evaluated on faithfulness, completeness, and correctness, each scored on a 0 to 1 scale by an LLM judge. Faithfulness measured whether the answer was grounded in the retrieved context; correctness measured factual accuracy against a gold answer; completeness measured whether the response addressed all parts of the question. Completeness proved the hardest to score consistently, as the judge's assessment varied depending on how exhaustive the gold answer was and how broadly the question could be interpreted.

Gold answers were synthesised using an LLM grounded in the relevant source chunks rather than written manually. For factual questions with a single correct answer, such as fee amounts or specific dates, this produced reliable reference answers. For open-ended policy questions, the synthesised gold answer represented one valid interpretation of the source text, and responses covering the same policy through different phrasing or emphasis were sometimes scored as incomplete rather than correct. Defining a ground truth that captures the range of acceptable answers for policy questions was not straightforward within an automated evaluation pipeline.

The evaluation framework covered single-turn queries only. Clarification flows and multi-turn context carryover were tested manually through targeted interaction scripts rather than as part of the 89-question evaluation set. This meant that some failure modes specific to multi-turn behaviour, such as repeated clarification prompts and incomplete slot resolution, were not captured in the reported metrics.

Responses that included plausible but unsupported details presented a scoring challenge. The faithfulness metric flagged answers where the model introduced content not present in the retrieved context, but a low faithfulness score could reflect either a genuine hallucination or a case where

relevant information existed in the source but was not retrieved. Distinguishing between a retrieval gap and a generation error required manual inspection and was not automated.

6.4 Collaboration Challenges

Connecting both group frontends to each other's backends required agreeing on a shared API contract mid-project. The request and response schemas diverged in small but breaking ways, particularly around how clarification state was serialized and whether source URLs were returned as a flat list or a nested object. Aligning these required several coordination rounds and a backend patch to ensure each chatbot only consumed the clarification state belonging to its own pipeline rather than the other group's

The 89 question evaluation required both teams to agree on question categories, scoring rubrics, and expected answers before testing began. Independently scored results occasionally differed on partial credit cases, and reconciling those disagreements during joint review sessions in the final week added time pressure to an already tight evaluation window.

7. Key Insights

7.1 System Insights

Neither lexical nor semantic retrieval is sufficient in isolation. BM25 handles exact terminology but fails on conversational phrasing; dense retrieval handles semantic intent but degrades on entity-specific queries like school names and fee categories. The hybrid approach is not an optimization, it is a necessity for this domain.

Chunk quality directly determines response quality. A correctly retrieved but poorly chunked document produces incomplete or misleading answers regardless of how good the retrieval pipeline is. Getting chunking right is as important as getting retrieval right.

Richer metadata enables more precise retrieval. Fields like school, level, term, and fee type allowed slot-filling to narrow the search space before retrieval ran, which reduced noise in the candidate set and improved answer precision.

Routing, slot-filling, clarification, and query augmentation collectively reduce retrieval scope before the search runs. Each layer filters out irrelevant candidates earlier in the pipeline, making retrieval faster and more targeted than running broad unfiltered queries against the full index.

Keeping the backend stateless by passing all session context with each request allowed both groups' frontends to connect to either backend with only schema alignment required, no structural changes needed. This simplified cross-group integration significantly.

7.2 Evaluation Insights

The ES-based hybrid approach demonstrated faster query resolution than pure RAG approaches, as structured slot-filling and domain filtering reduced the candidate pool before retrieval rather than relying entirely on the LLM to resolve ambiguity post-retrieval.

Out-of-domain queries are not always rejected cleanly. Semantic routing occasionally pulled contact-domain results for queries that had no genuine contact intent, because embedding similarity found surface-level matches in the directory index. Hard boundary logic is needed alongside semantic routing.

Hit Rate@k is a more meaningful metric than Recall@k for this use case. Retrieving at least one correct chunk matters more than retrieving every possible relevant chunk, and recall is naturally suppressed when gold sets are large. Hit rate better reflects whether the system can answer the question at all.

Blind testing consistently produced lower pass rates than structured evaluation, revealing that structured test sets overestimate real-world performance. Evaluation on unseen conversational queries should be treated as the primary performance benchmark

7.3 Auto-RAG Insights

NotebookLM outperformed the hybrid system on most categories, which is expected given its broader document grounding and more capable general-purpose model. The meaningful finding is where it did not: the hybrid system outperformed NotebookLM on Contacts, the one domain where structured indexed data with field-level matching has a direct advantage over general document reading. This suggests that for domains with highly structured data and entity-specific queries, purpose-built retrieval pipelines remain competitive against automated approaches.

8. Conclusion & Future Work

8.1 Conclusion

This project developed a Hybrid University Information Chatbot for Illinois Institute of Technology that combines Elasticsearch-based retrieval with GPT-4o for natural language answer generation across four domains: academic policies, tuition and fees, the academic calendar, and department contacts. The system demonstrates that domain-specialized hybrid retrieval, combining BM25 lexical search with kNN vector search fused via Reciprocal Rank Fusion and reranked by a cross-encoder, produces accurate and grounded answers that a pure RAG approach cannot match on structured, entity-specific queries. Evaluation across 89 questions spanning nine categories showed hit rates between 81% and 100%, with the system achieving a mean answer quality score of 4.28 out of 5 and a perfect answer rate of 66.3%. The structured retrieval pipeline, supported by slot-filling, query expansion, and clarification dialogs, consistently outperformed pure RAG on domains requiring precise fact retrieval such as tuition rates, calendar dates, and contact information.

Limitations remain in three areas. Contacts retrieval is constrained by the completeness of the scraped directory data rather than the retrieval pipeline itself. Out-of-scope queries are not consistently rejected, with semantic routing occasionally surfacing irrelevant results for queries outside the system's knowledge boundaries. Multi-turn follow-up handling, while improved through iteration, remains the weakest conversational capability at 35% in session evaluation. These limitations inform the directions for future work described in the following section.

8.2 Future Work

The most immediate gap is data freshness. All four indices are built from static datasets, so tuition rates, calendar events, and directory entries need to be updated each semester. A scheduled re-indexing pipeline would address this without manual effort.

Beyond data freshness, three further directions stand out. Multi-topic queries were the weakest category in evaluation, and better merging of results from parallel domain searches before generation would improve answer quality there. Domain coverage could be extended to housing, financial aid, and graduate admissions using the existing pipeline structure. On the frontend, linking each part of a response to its source page would help users verify answers directly, which matters most for time-sensitive information like deadlines and fee amounts.

References

- [1] C. Gormley and Z. Tong, *Elasticsearch: The Definitive Guide*. O'Reilly Media, 2015.
- [2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [3] L. Wang et al., "Text Embeddings by Weakly-Supervised Contrastive Pre-training," arXiv:2212.03533, 2022.
- [4] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP*, 2020.
- [5] J. Lin, R. Nogueira, and A. Yates, "Pretrained Transformers for Text Ranking: BERT and Beyond," *Synthesis Lectures on Human Language Technologies*, 2021.
- [6] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *Proc. SIGIR*, 2009.
- [8] T. Nguyen et al., "MS MARCO: A Human Generated Machine Reading Comprehension Dataset," in *NeurIPS Workshop on Cognitive Computation*, 2016.
- [9] S. Ramírez, "FastAPI Documentation," 2023. [Online]. Available: <https://fastapi.tiangolo.com>